

# Understanding Transformers: A Step-by-Step Math Example

## Step 1 — Defining our Dataset

The dataset used for creating ChatGPT is **570 GB**. But we will be using a very small dataset to perform numerical calculations visually.

### Dataset (corpus)

I drink and I know things.  
When you play the game of thrones, you win or you die.  
The true enemy won't wait out the storm, He brings the storm.

Our entire dataset contains only three sentences, all of which are dialogues taken from a TV show. Although our dataset is cleaned, in real-world scenarios cleaning a dataset requires a significant amount of effort.

## Step 2 — Finding Vocab Size

The vocabulary size determines the total number of **unique words** in our dataset. It can be calculated using the below formula, where **N** is the total number of words in our dataset.

$$vocab\ size = count(set(N))$$

In order to find N, we need to break our dataset into individual words.

### Dataset (Corpus)

I drink and I know things.  
When you play the game of thrones, you win or you die.  
The true enemy won't wait out the storm, He brings the storm.

$$N = \begin{bmatrix} I, \text{drink}, \text{and}, I, \text{Know}, \text{things}, \\ \text{When}, \text{you}, \text{play}, \text{the}, \text{game}, \text{of}, \text{thrones}, \text{you}, \text{win}, \text{or}, \text{you}, \text{die}, \\ \text{The}, \text{true}, \text{enemy}, \text{won't}, \text{wait}, \text{out}, \text{the}, \text{storm}, \text{He}, \text{brings}, \text{the}, \text{storm} \end{bmatrix}$$

After obtaining N, we perform a set operation to remove duplicates, and then we can count the unique words to determine the vocabulary size. Therefore, the vocabulary size is **23**, as there are **23** unique words in our dataset.

$$vocab\ size = count(set(N))$$

$$\begin{aligned} &\rightarrow set( \begin{bmatrix} I, \text{drink}, \text{and}, I, \text{Know}, \text{things}, \\ \text{When}, \text{you}, \text{play}, \text{the}, \text{game}, \text{of}, \text{thrones}, \text{you}, \text{win}, \text{or}, \text{you}, \text{die}, \\ \text{The}, \text{true}, \text{enemy}, \text{won't}, \text{wait}, \text{out}, \text{the}, \text{storm}, \text{He}, \text{brings}, \text{the}, \text{storm} \end{bmatrix} ) \\ &\rightarrow count( \begin{bmatrix} I, \text{drink}, \text{and}, \text{Know}, \text{things}, \text{When}, \text{you}, \text{play}, \text{the}, \text{game}, \text{of}, \\ \text{thrones}, \text{win}, \text{or}, \text{die}, \text{true}, \text{enemy}, \text{won't}, \text{wait}, \text{out}, \text{storm}, \text{He}, \\ \text{brings} \end{bmatrix} ) \\ &\rightarrow = 23 \end{aligned}$$

## Step 3 — Encoding

Now, we need to assign a unique number to each unique word.

|     |       |        |       |      |       |         |     |      |       |        |      |
|-----|-------|--------|-------|------|-------|---------|-----|------|-------|--------|------|
| 1   | 2     | 3      | 4     | 5    | 6     | 7       | 8   | 9    | 10    | 11     | 12   |
| I   | drink | things | Know  | When | won't | play    | out | true | storm | brings | game |
| 13  | 14    | 15     | 16    | 17   | 18    | 19      | 20  | 21   | 22    | 23     |      |
| the | win   | of     | enemy | you  | wait  | thrones | and | or   | die   | He     |      |

#### Step 4 — Calculating Embedding

Let's select a sentence from our corpus that will be processed in our transformer architecture.

|     |       |        |       |      |       |         |     |      |       |        |      |
|-----|-------|--------|-------|------|-------|---------|-----|------|-------|--------|------|
| 1   | 2     | 3      | 4     | 5    | 6     | 7       | 8   | 9    | 10    | 11     | 12   |
| I   | drink | things | Know  | When | won't | play    | out | true | storm | brings | game |
| 13  | 14    | 15     | 16    | 17   | 18    | 19      | 20  | 21   | 22    | 23     |      |
| the | win   | of     | enemy | you  | wait  | thrones | and | or   | die   | He     |      |



We have selected our input, and we need to find an embedding vector for it. The original paper uses a **512-dimensional embedding vector** for each input word. Since, for our case, we need to work with a smaller dimension of embedding vector to visualize how the calculation is taking place. So, we will be using a dimension of 6 for the embedding vector.

|  | When | you | play | game | of | thrones |
|--|------|-----|------|------|----|---------|
|  | 5    | 17  | 7    | 12   | 15 | 19      |

|  | e1   | e2   | e3   | e4   | e5   | e6   |
|--|------|------|------|------|------|------|
|  | 0.79 | 0.38 | 0.01 | 0.12 | 0.88 | 0.6  |
|  | 0.6  | 0.12 | 0.51 | 0.6  | 0.41 | 0.33 |
|  | 0.96 | 0.06 | 0.27 | 0.65 | 0.79 | 0.75 |
|  | 0.64 | 0.79 | 0.31 | 0.22 | 0.62 | 0.48 |
|  | 0.97 | 0.9  | 0.56 | 0.07 | 0.5  | 0.94 |
|  | 0.2  | 0.74 | 0.59 | 0.37 | 0.7  | 0.21 |

These values of the embedding vector are between 0 and 1 and are filled randomly in the beginning. They will later be updated as our transformer starts understanding the meanings among the words.

#### Step 5 — Calculating Positional Embedding

Now we need to find positional embeddings for our input. There are two formulas for positional embedding depending on the position of the  $i$ th value of that embedding vector for each word.

Embedding vector for any word

|               |
|---------------|
| even position |
| odd position  |
| even position |
| odd position  |
| even position |

...

For even position

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

For odd position

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

As you do know, our input sentence is “when you play the game of thrones” and the starting word is “when” with a starting index (POS) value is 0, having a dimension (d) of 6. For i from 0 to 5, we calculate the positional embedding for our first word of the input sentence.

|      |
|------|
| When |
| 5    |

| i | e1   | Position | Formula                   | p1 |
|---|------|----------|---------------------------|----|
| 0 | 0.79 | Even     | $\sin(0/10000^{(2*0/6)})$ | 0  |
| 1 | 0.6  | Odd      | $\cos(0/10000^{(2*1/6)})$ | 1  |
| 2 | 0.96 | Even     | $\sin(0/10000^{(2*2/6)})$ | 0  |
| 3 | 0.64 | Odd      | $\cos(0/10000^{(2*3/6)})$ | 1  |
| 4 | 0.97 | Even     | $\sin(0/10000^{(2*4/6)})$ | 0  |
| 5 | 0.2  | Odd      | $\cos(0/10000^{(2*5/6)})$ | 1  |

d (dim) 6

POS 0

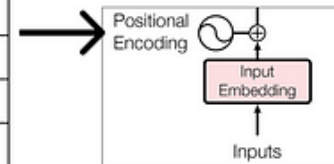
Similarly, we can calculate positional embedding for all the words in our input sentence.

| When | you | play | game | of | thrones |
|------|-----|------|------|----|---------|
| 5    | 17  | 7    | 12   | 15 | 19      |

| i | p1 | p2     | p3     | p4     | p5      | p6      |
|---|----|--------|--------|--------|---------|---------|
| 0 | 0  | 0.8415 | 0.9093 | 0.1411 | -0.7568 | -0.9589 |
| 1 | 1  | 0.0464 | 0.9957 | 0.1388 | 0.1846  | 0.9732  |
| 2 | 0  | 0.0022 | 0.0043 | 0.0065 | 0.0086  | 0.0108  |
| 3 | 1  | 0.0001 | 1      | 0.0003 | 0.0004  | 1       |
| 4 | 0  | 0      | 0      | 0      | 0       | 0       |
| 5 | 1  | 0      | 1      | 0      | 0       | 1       |

d (dim) 6 6 6 6 6 6

POS 0 1 2 3 4 5



## Step 6 — Concatenating Positional and Word Embeddings

After calculating positional embedding, we need to add word embeddings and positional embeddings.

|      |     |      |      |    |         |
|------|-----|------|------|----|---------|
| When | you | play | game | of | thrones |
| 5    | 17  | 7    | 12   | 15 | 19      |

Position Embedding Matrix

| p1 | p2     | p3     | p4     | p5      | p6      |
|----|--------|--------|--------|---------|---------|
| 0  | 0.8415 | 0.9093 | 0.1411 | -0.7568 | -0.9589 |
| 1  | 0.0464 | 0.9957 | 0.1388 | 0.1846  | 0.9732  |
| 0  | 0.0022 | 0.0043 | 0.0065 | 0.0086  | 0.0108  |
| 1  | 0.0001 | 1      | 0.0003 | 0.0004  | 1       |
| 0  | 0      | 0      | 0      | 0       | 0       |
| 1  | 0      | 1      | 0      | 0       | 1       |

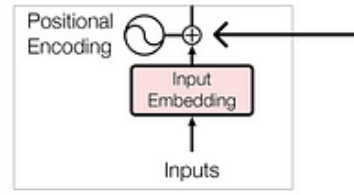
+

Word Embedding Matrix

| e1   | e2   | e3   | e4   | e5   | e6   |
|------|------|------|------|------|------|
| 0.79 | 0.38 | 0.01 | 0.12 | 0.88 | 0.6  |
| 0.6  | 0.12 | 0.51 | 0.6  | 0.41 | 0.33 |
| 0.96 | 0.06 | 0.27 | 0.65 | 0.79 | 0.75 |
| 0.64 | 0.79 | 0.31 | 0.22 | 0.62 | 0.48 |
| 0.97 | 0.9  | 0.56 | 0.07 | 0.5  | 0.94 |
| 0.2  | 0.74 | 0.59 | 0.37 | 0.7  | 0.21 |

=

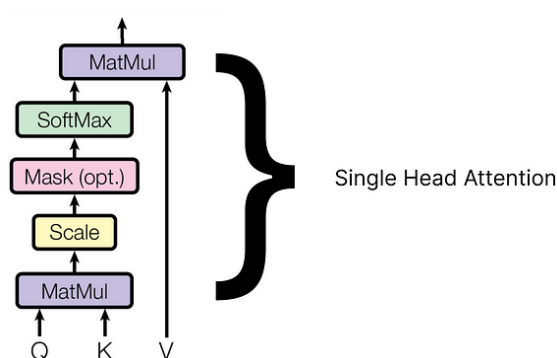
| ep1  | ep2  | ep3  | ep4  | ep5  | ep6   |
|------|------|------|------|------|-------|
| 0.79 | 1.22 | 0.92 | 0.26 | 0.12 | -0.36 |
| 1.6  | 0.17 | 1.51 | 0.74 | 0.59 | 1.3   |
| 0.96 | 0.06 | 0.27 | 0.66 | 0.8  | 0.76  |
| 1.64 | 0.79 | 1.31 | 0.22 | 0.62 | 1.48  |
| 0.97 | 0.9  | 0.56 | 0.07 | 0.5  | 0.94  |
| 1.2  | 0.74 | 1.59 | 0.37 | 0.7  | 1.21  |



This resultant matrix from combining both matrices (**Word embedding matrix** and **positional embedding matrix**) will be considered as an input to the encoder part.

### Step 7 — Multi Head Attention

A multi-head attention is comprised of many single-head attentions. It is up to us how many single heads we need to combine. For example, LLaMA LLM from Meta has used 32 single heads in the encoder architecture. Below is the illustrated diagram of how a single-head attention looks like.



There are three inputs: **query**, **key**, and **value**. Each of these matrices is obtained by multiplying a different set of weights matrix from the **Transpose** of same matrix that we computed earlier by adding the word embedding and positional embedding matrix.

Let's say, for computing the query matrix, the set of weights matrix must have the number of rows the same as the number of columns of the transpose matrix, while the columns of the weights matrix can be any; for example, we suppose 4 columns in our weights matrix. The values in the weights matrix are between 0 and 1 randomly, which will later be updated when our transformer starts learning the meaning of these words.

Word Embedding + Positional Embedding

|         |       |      |      |      |      |      |
|---------|-------|------|------|------|------|------|
| When    | 0.79  | 1.6  | 0.96 | 1.64 | 0.97 | 1.2  |
| you     | 1.22  | 0.17 | 0.06 | 0.79 | 0.9  | 0.74 |
| play    | 0.92  | 1.51 | 0.27 | 1.31 | 0.56 | 1.59 |
| game    | 0.26  | 0.74 | 0.66 | 0.22 | 0.07 | 0.37 |
| of      | 0.12  | 0.59 | 0.8  | 0.62 | 0.5  | 0.7  |
| thrones | -0.36 | 1.3  | 0.76 | 1.48 | 0.94 | 1.21 |

6 x 6

Linear weights for query

|      |      |      |      |
|------|------|------|------|
| 0.52 | 0.45 | 0.91 | 0.69 |
| 0.05 | 0.85 | 0.37 | 0.83 |
| 0.49 | 0.1  | 0.56 | 0.61 |
| 0.71 | 0.64 | 0.4  | 0.14 |
| 0.76 | 0.27 | 0.92 | 0.67 |
| 0.85 | 0.56 | 0.57 | 0.07 |

6 x 4

X

Similarly, we can compute the **key** and **value** matrices using the same procedure, but the values in the weights matrix must be different for both.

Word Embedding + Positional Embedding

|         |       |      |      |      |      |      |
|---------|-------|------|------|------|------|------|
| When    | 0.79  | 1.6  | 0.96 | 1.64 | 0.97 | 1.2  |
| you     | 1.22  | 0.17 | 0.06 | 0.79 | 0.9  | 0.74 |
| play    | 0.92  | 1.51 | 0.27 | 1.31 | 0.56 | 1.59 |
| game    | 0.26  | 0.74 | 0.66 | 0.22 | 0.07 | 0.37 |
| of      | 0.12  | 0.59 | 0.8  | 0.62 | 0.5  | 0.7  |
| thrones | -0.36 | 1.3  | 0.76 | 1.48 | 0.94 | 1.21 |

6 x 6

Linear weights for key

|      |      |      |      |
|------|------|------|------|
| 0.74 | 0.57 | 0.21 | 0.73 |
| 0.55 | 0.16 | 0.9  | 0.17 |
| 0.25 | 0.74 | 0.8  | 0.98 |
| 0.8  | 0.73 | 0.2  | 0.31 |
| 0.37 | 0.96 | 0.42 | 0.08 |
| 0.28 | 0.41 | 0.87 | 0.86 |

6 x 4

X

Word Embedding + Positional Embedding

|         |       |      |      |      |      |      |
|---------|-------|------|------|------|------|------|
| When    | 0.79  | 1.6  | 0.96 | 1.64 | 0.97 | 1.2  |
| you     | 1.22  | 0.17 | 0.06 | 0.79 | 0.9  | 0.74 |
| play    | 0.92  | 1.51 | 0.27 | 1.31 | 0.56 | 1.59 |
| game    | 0.26  | 0.74 | 0.66 | 0.22 | 0.07 | 0.37 |
| of      | 0.12  | 0.59 | 0.8  | 0.62 | 0.5  | 0.7  |
| thrones | -0.36 | 1.3  | 0.76 | 1.48 | 0.94 | 1.21 |

6 x 6

Linear weights for value

|      |      |      |      |
|------|------|------|------|
| 0.62 | 0.07 | 0.7  | 0.95 |
| 0.2  | 0.97 | 0.61 | 0.35 |
| 0.57 | 0.8  | 0.61 | 0.5  |
| 0.67 | 0.35 | 0.98 | 0.54 |
| 0.47 | 0.83 | 0.34 | 0.94 |
| 0.6  | 0.69 | 0.13 | 0.98 |

6 x 4

X

So, after multiplying matrices, the resultant **query**, **key**, and **values** are obtained:

Query

|      |      |      |      |
|------|------|------|------|
| 3.88 | 3.8  | 4.08 | 3.42 |
| 2.55 | 1.86 | 2.77 | 1.78 |
| 3.39 | 3.6  | 3.49 | 2.72 |
| 1.02 | 1.18 | 1.24 | 1.3  |
| 1.9  | 1.56 | 1.88 | 1.53 |
| 3.04 | 2.9  | 2.73 | 2.22 |

6 x 4

key

|      |      |      |      |
|------|------|------|------|
| 3.71 | 4.04 | 4.15 | 3.41 |
| 2.18 | 2.51 | 1.64 | 1.93 |
| 3.28 | 3.11 | 3.65 | 3.01 |
| 1.07 | 1.13 | 1.64 | 1.35 |
| 1.49 | 1.97 | 2.14 | 1.81 |
| 2.51 | 3.04 | 3.45 | 2.28 |

6 x 4

value

|      |      |      |      |
|------|------|------|------|
| 3.88 | 3.8  | 4.08 | 3.42 |
| 2.55 | 1.86 | 2.77 | 1.78 |
| 3.39 | 3.6  | 3.49 | 2.72 |
| 1.02 | 1.18 | 1.24 | 1.3  |
| 1.9  | 1.56 | 1.88 | 1.53 |
| 3.04 | 2.9  | 2.73 | 2.22 |

6 x 4

Now that we have all three matrices, let's start calculating single-head attention step by step.

Query

|      |      |      |      |
|------|------|------|------|
| 3.88 | 3.8  | 4.08 | 3.42 |
| 2.55 | 1.86 | 2.77 | 1.78 |
| 3.39 | 3.6  | 3.49 | 2.72 |
| 1.02 | 1.18 | 1.24 | 1.3  |
| 1.9  | 1.56 | 1.88 | 1.53 |
| 3.04 | 2.9  | 2.73 | 2.22 |

6 x 4

Transpose ( Key )

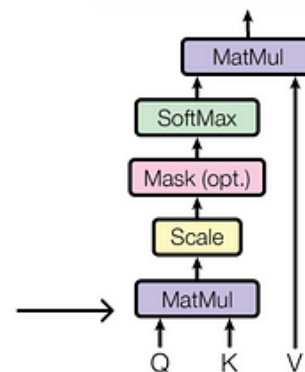
|      |      |      |      |      |      |
|------|------|------|------|------|------|
| 3.71 | 2.18 | 3.28 | 1.07 | 1.49 | 2.51 |
| 4.04 | 2.51 | 3.11 | 1.13 | 1.97 | 3.04 |
| 4.15 | 1.64 | 3.65 | 1.64 | 2.14 | 3.45 |
| 3.41 | 1.93 | 3.01 | 1.35 | 1.81 | 2.28 |

4 x 6

X

=

|         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|
| 58.341  | 31.2882 | 49.7306 | 19.7538 | 28.1886 | 43.1644 |
| 34.5402 | 18.2058 | 29.6169 | 11.7761 | 16.6133 | 25.6698 |
| 50.8796 | 27.3994 | 43.2409 | 17.0909 | 24.5349 | 37.695  |
| 18.1304 | 9.728   | 15.4544 | 6.2134  | 8.851   | 13.3894 |
| 26.3707 | 14.0937 | 22.5509 | 8.9445  | 12.6967 | 19.4858 |
| 41.8941 | 22.668  | 35.6369 | 14.004  | 20.103  | 30.9265 |



For scaling the resultant matrix, we have to reuse the dimension of our embedding vector, which is 6.

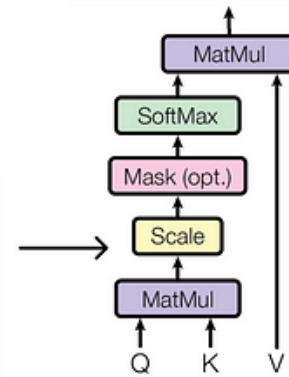


|         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|
| 58.341  | 31.2882 | 49.7306 | 19.7538 | 28.1886 | 43.1644 |
| 34.5402 | 18.2058 | 29.6169 | 11.7761 | 16.6133 | 25.6698 |
| 50.8796 | 27.3994 | 43.2409 | 17.0909 | 24.5349 | 37.695  |
| 18.1304 | 9.728   | 15.4544 | 6.2134  | 8.851   | 13.3894 |
| 26.3707 | 14.0937 | 22.5509 | 8.9445  | 12.6967 | 19.4858 |
| 41.8941 | 22.668  | 35.6369 | 14.004  | 20.103  | 30.9265 |

$\sqrt{d_k}$  where d (dimension) is 6

=

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| 23.81721 | 12.77409 | 20.30219 | 8.062904 | 11.50852 | 17.62    |
| 14.1009  | 7.434201 | 12.09231 | 4.809165 | 6.781004 | 10.47973 |
| 20.77167 | 11.186   | 17.65266 | 6.976963 | 10.01433 | 15.39096 |
| 7.401542 | 3.972256 | 6.307436 | 2.535222 | 3.612997 | 5.466445 |
| 10.76551 | 5.752218 | 9.205999 | 3.64974  | 5.184753 | 7.956759 |
| 17.10152 | 9.254989 | 14.54997 | 5.715476 | 8.205791 | 12.62712 |



The next step of **masking is optional**, and we won't be calculating it. Masking is like telling the model to focus only on what's happened before a certain point and not peek into the future while figuring out the importance of different words in a sentence. It helps the model understand things in a step-by-step manner, without cheating by looking ahead. So now we will be applying the **softmax** operation on our scaled resultant matrix.

|       |       |       |      |       |       |
|-------|-------|-------|------|-------|-------|
| 23.82 | 12.77 | 20.3  | 8.06 | 11.51 | 17.62 |
| 14.1  | 7.43  | 12.09 | 4.81 | 6.78  | 10.48 |
| 20.77 | 11.19 | 17.65 | 6.98 | 10.01 | 15.39 |
| 7.4   | 3.97  | 6.31  | 2.54 | 3.61  | 5.47  |
| 10.77 | 5.75  | 9.21  | 3.65 | 5.18  | 7.96  |
| 17.1  | 9.25  | 14.55 | 5.72 | 8.21  | 12.63 |

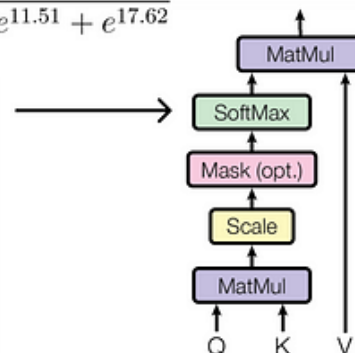
$$\text{SoftMax}$$

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

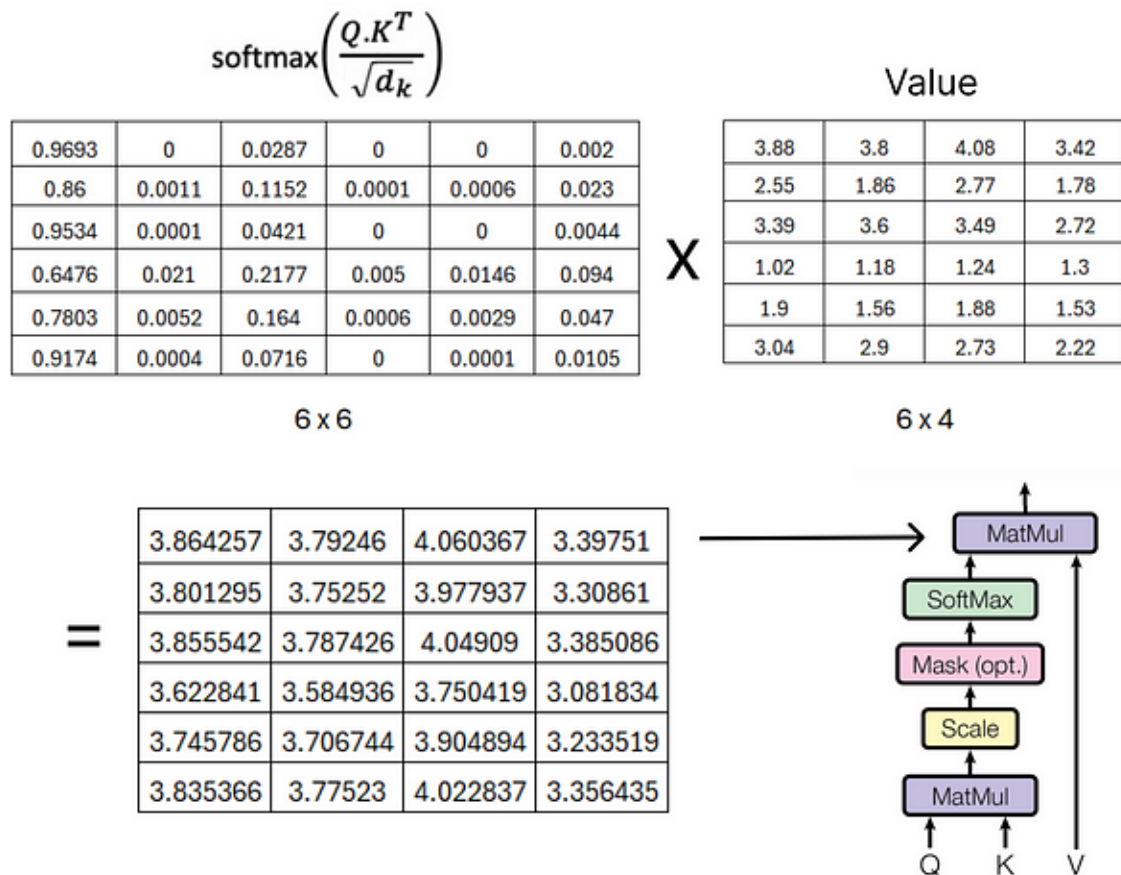
$$\text{softmax}(23.82) = \frac{e^{23.82}}{e^{23.82} + e^{12.77} + e^{20.3} + e^{8.06} + e^{11.51} + e^{17.62}}$$

=

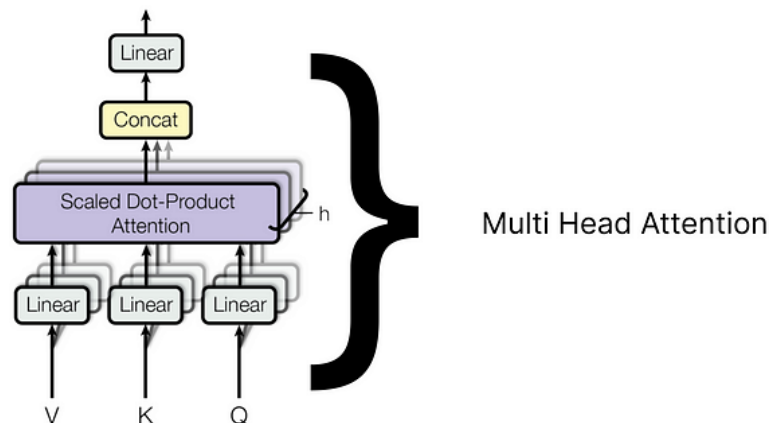
|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| 0.9693 | 0      | 0.0287 | 0      | 0      | 0.002  |
| 0.86   | 0.0011 | 0.1152 | 0.0001 | 0.0006 | 0.023  |
| 0.9534 | 0.0001 | 0.0421 | 0      | 0      | 0.0044 |
| 0.6476 | 0.021  | 0.2177 | 0.005  | 0.0146 | 0.094  |
| 0.7803 | 0.0052 | 0.164  | 0.0006 | 0.0029 | 0.047  |
| 0.9174 | 0.0004 | 0.0716 | 0      | 0.0001 | 0.0105 |



Doing the final multiplication step to obtain the resultant matrix from single-head attention.



We have calculated single-head attention, while multi-head attention comprises many single-head attentions, as I stated earlier. Below is a visual of how it looks like:

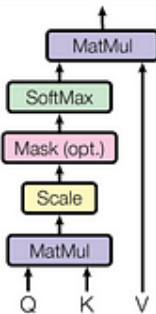


Each single-head attention has three inputs: **query**, **key**, and **value**, and each three have a different set of weights. Once all single-head attentions output their resultant matrices, they will all be concatenated, and the final concatenated matrix is once again transformed linearly by multiplying it with a set of weights matrix initialized with random values, which will later get updated when the transformer starts training. Since, in our case, we are considering a single-head attention, but this is how it looks if we are working with multi-head attention.

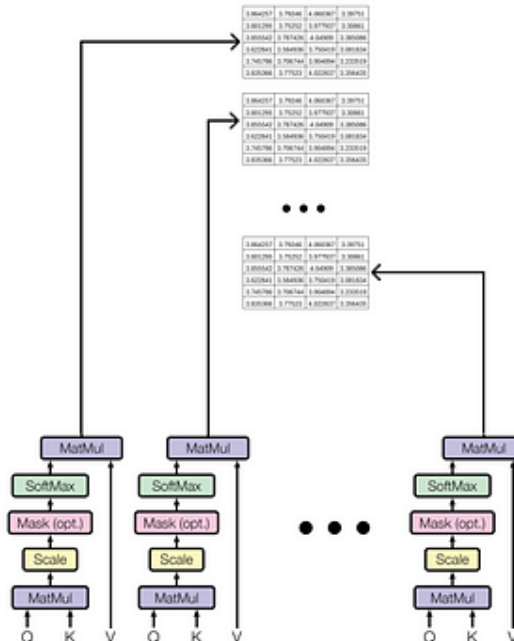


## Single Head Attention Our Case

|          |          |          |          |
|----------|----------|----------|----------|
| 3.864257 | 3.79246  | 4.060367 | 3.39751  |
| 3.801295 | 3.75252  | 3.977937 | 3.30861  |
| 3.855542 | 3.787426 | 4.04909  | 3.385086 |
| 3.622841 | 3.584936 | 3.750419 | 3.081834 |
| 3.745786 | 3.706744 | 3.904894 | 3.233519 |
| 3.835366 | 3.77523  | 4.022837 | 3.356435 |



## Multi Head Attention (N Heads) Real world Case Concatenation



In either case, whether it's single-head or multi-head attention, the resultant matrix needs to be once again transformed linearly by multiplying a set of weights matrix.

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \times \text{Value}$$

|      |      |      |      |
|------|------|------|------|
| 3.86 | 3.79 | 4.06 | 3.4  |
| 3.8  | 3.75 | 3.98 | 3.31 |
| 3.86 | 3.79 | 4.05 | 3.39 |
| 3.62 | 3.58 | 3.75 | 3.08 |
| 3.75 | 3.71 | 3.9  | 3.23 |
| 3.84 | 3.78 | 4.02 | 3.36 |

6 x 4

X

Linear weights  
columns length must be  
(embedding+positional) matrix columns length

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| 0.8  | 0.34 | 0.45 | 0.54 | 0.07 | 0.53 |
| 0.85 | 0.74 | 0.78 | 0.5  | 0.75 | 0.55 |
| 0.53 | 0.81 | 0.55 | 0.59 | 0.49 | 0.14 |
| 0.7  | 0.6  | 0.12 | 0.42 | 0.29 | 0.87 |

4 x 6

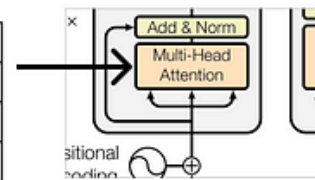
$$=$$

|       |      |      |      |      |      |
|-------|------|------|------|------|------|
| 10.84 | 9.45 | 7.33 | 7.8  | 6.09 | 7.66 |
| 10.65 | 9.28 | 7.22 | 7.67 | 5.99 | 7.51 |
| 10.83 | 9.43 | 7.33 | 7.79 | 6.08 | 7.65 |
| 10.08 | 8.77 | 6.85 | 7.25 | 5.67 | 7.09 |
| 10.48 | 9.12 | 7.11 | 7.54 | 5.89 | 7.38 |
| 10.77 | 9.38 | 7.29 | 7.75 | 6.05 | 7.6  |

Make sure the linear set of weights matrix number of columns must be equal to the matrix that we computed earlier (**word embedding + positional embedding**) matrix number of columns, because the next step, we will be adding the resultant normalized matrix with (**word embedding + positional embedding**) matrix.

Output of Multi Head attention

|       |      |      |      |      |      |
|-------|------|------|------|------|------|
| 10.84 | 9.45 | 7.33 | 7.8  | 6.09 | 7.66 |
| 10.65 | 9.28 | 7.22 | 7.67 | 5.99 | 7.51 |
| 10.83 | 9.43 | 7.33 | 7.79 | 6.08 | 7.65 |
| 10.08 | 8.77 | 6.85 | 7.25 | 5.67 | 7.09 |
| 10.48 | 9.12 | 7.11 | 7.54 | 5.89 | 7.38 |
| 10.77 | 9.38 | 7.29 | 7.75 | 6.05 | 7.6  |



6 x 6

As we have computed the resultant matrix for multi-head attention, next, we will be working on adding and normalizing step.

### Step 8 — Adding and Normalizing

Once we obtain the resultant matrix from multi-head attention, we have to add it to our original matrix. Let's do it first.

Word Embedding + Positional Embedding

|         |       |      |      |      |      |      |
|---------|-------|------|------|------|------|------|
| When    | 0.79  | 1.6  | 0.96 | 1.64 | 0.97 | 1.2  |
| you     | 1.22  | 0.17 | 0.06 | 0.79 | 0.9  | 0.74 |
| play    | 0.92  | 1.51 | 0.27 | 1.31 | 0.56 | 1.59 |
| game    | 0.26  | 0.74 | 0.66 | 0.22 | 0.07 | 0.37 |
| of      | 0.12  | 0.59 | 0.8  | 0.62 | 0.5  | 0.7  |
| thrones | -0.36 | 1.3  | 0.76 | 1.48 | 0.94 | 1.21 |

6 x 6

+

=

Output of Multi Head attention

|       |      |      |      |      |      |
|-------|------|------|------|------|------|
| 10.84 | 9.45 | 7.33 | 7.8  | 6.09 | 7.66 |
| 10.65 | 9.28 | 7.22 | 7.67 | 5.99 | 7.51 |
| 10.83 | 9.43 | 7.33 | 7.79 | 6.08 | 7.65 |
| 10.08 | 8.77 | 6.85 | 7.25 | 5.67 | 7.09 |
| 10.48 | 9.12 | 7.11 | 7.54 | 5.89 | 7.38 |
| 10.77 | 9.38 | 7.29 | 7.75 | 6.05 | 7.6  |

6 x 6

|       |       |      |      |      |      |
|-------|-------|------|------|------|------|
| 11.63 | 11.05 | 8.29 | 9.44 | 7.06 | 8.86 |
| 11.87 | 9.45  | 7.28 | 8.46 | 6.89 | 8.25 |
| 11.75 | 10.94 | 7.6  | 9.1  | 6.64 | 9.24 |
| 10.34 | 9.51  | 7.51 | 7.47 | 5.74 | 7.46 |
| 10.6  | 9.71  | 7.91 | 8.16 | 6.39 | 8.08 |
| 10.41 | 10.68 | 8.05 | 9.23 | 6.99 | 8.81 |

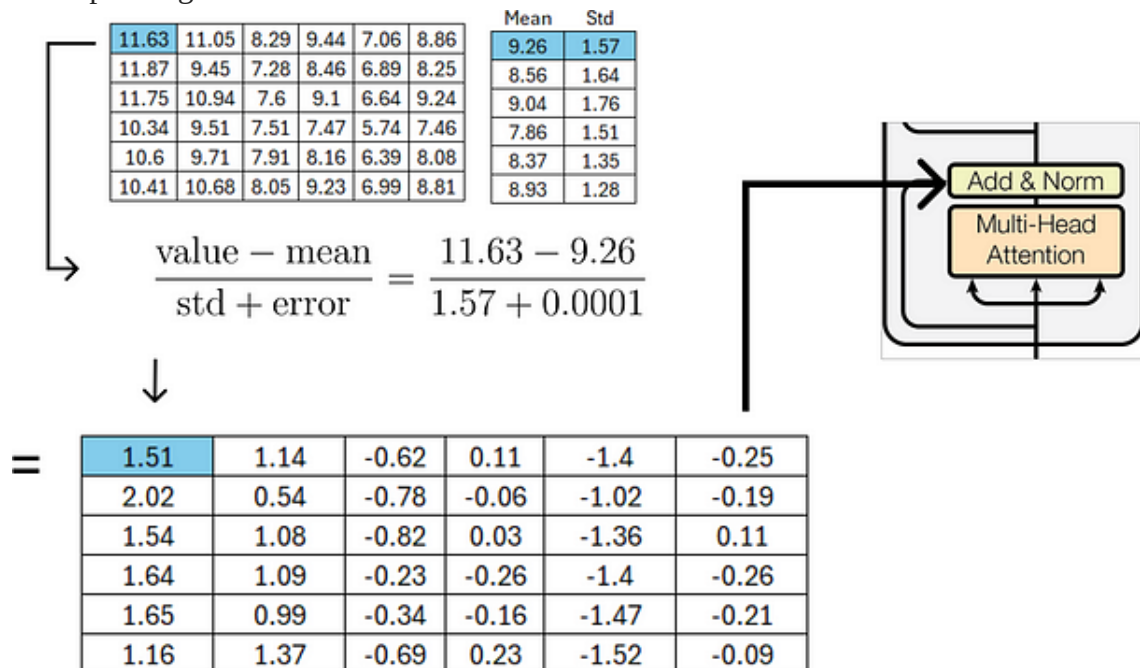
To normalize the above matrix, we need to compute the mean and standard deviation row-wise for each row.

$$mean = \frac{\sum_{i=1}^N X_i}{N} \quad standard\ dev. = \sqrt{\frac{\sum_{i=1}^N (X_i - \mu)^2}{N}}$$

Row Wise Implementation

|       |       |      |      |      |      |   | Mean | Standard Deviation |
|-------|-------|------|------|------|------|---|------|--------------------|
| 11.63 | 11.05 | 8.29 | 9.44 | 7.06 | 8.86 | → | 9.26 | 1.57               |
| 11.87 | 9.45  | 7.28 | 8.46 | 6.89 | 8.25 | → | 8.56 | 1.64               |
| 11.75 | 10.94 | 7.6  | 9.1  | 6.64 | 9.24 | → | 9.04 | 1.76               |
| 10.34 | 9.51  | 7.51 | 7.47 | 5.74 | 7.46 | → | 7.86 | 1.51               |
| 10.6  | 9.71  | 7.91 | 8.16 | 6.39 | 8.08 | → | 8.37 | 1.35               |
| 10.41 | 10.68 | 8.05 | 9.23 | 6.99 | 8.81 | → | 8.93 | 1.28               |

we subtract each value of the matrix by the corresponding row mean and divide it by the corresponding standard deviation.



Adding a small value of error prevents the denominator from being zero and avoids making the entire term infinity.

### Step 9 — Feed Forward Network

After normalizing the matrix, it will be processed through a feedforward network. We will be using a very basic network that contains only one linear layer and one ReLU activation function layer. This is how it looks like visually:

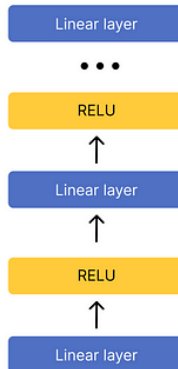
$$\text{ReLU}(x) = \max(0, x)$$

$$\text{Linear Layer} = X \cdot W + b$$

our case (one linear layer)



Real world case  
(multiple layers)



First, we need to calculate the linear layer by multiplying our last calculated matrix with a random set of weights matrix, which will be updated when the transformer starts learning, and adding the resultant matrix to a bias matrix that also contains random values.

Matrix after add and norm step

|      |      |       |       |       |       |
|------|------|-------|-------|-------|-------|
| 1.51 | 1.14 | -0.62 | 0.11  | -1.4  | -0.25 |
| 2.02 | 0.54 | -0.78 | -0.06 | -1.02 | -0.19 |
| 1.54 | 1.08 | -0.82 | 0.03  | -1.36 | 0.11  |
| 1.64 | 1.09 | -0.23 | -0.26 | -1.4  | -0.26 |
| 1.65 | 0.99 | -0.34 | -0.16 | -1.47 | -0.21 |
| 1.16 | 1.37 | -0.69 | 0.23  | -1.52 | -0.09 |

6 x 6

X

W

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| 0.5  | 0.05 | 0.97 | 0.22 | 0.56 | 0.02 |
| 0.17 | 0.52 | 0.63 | 0.48 | 0.06 | 0.6  |
| 0.53 | 0.87 | 0.47 | 0.1  | 0.31 | 0.79 |
| 0.83 | 0.58 | 0.38 | 0.09 | 0.64 | 0.25 |
| 0.81 | 0.85 | 0.74 | 0.35 | 0.31 | 0.53 |
| 0.25 | 0.31 | 0.22 | 0.77 | 0.57 | 0.85 |

6 x 6

$$X \cdot W$$

|      |      |       |      |      |      |
|------|------|-------|------|------|------|
| 0.49 | 1.07 | 0.84  | 0.14 | 0.22 | 0.7  |
| 0.24 | 1.26 | 1.11  | 0.12 | 0.46 | 0.97 |
| 0.53 | 1.18 | -0.82 | 0.39 | 0.33 | 0.59 |
| 0.53 | 0.97 | 0.98  | 0.15 | 0.16 | 0.52 |
| 0.56 | 1.11 | -0.87 | 0.11 | 0.2  | 0.64 |
| 0.62 | 1.02 | 0.61  | 0.26 | 0.14 | 0.52 |

6 x 6

+

Bias

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| b1   | b2   | b3   | b4   | b5   | b6   |
| 0.42 | 0.18 | 0.25 | 0.42 | 0.35 | 0.45 |

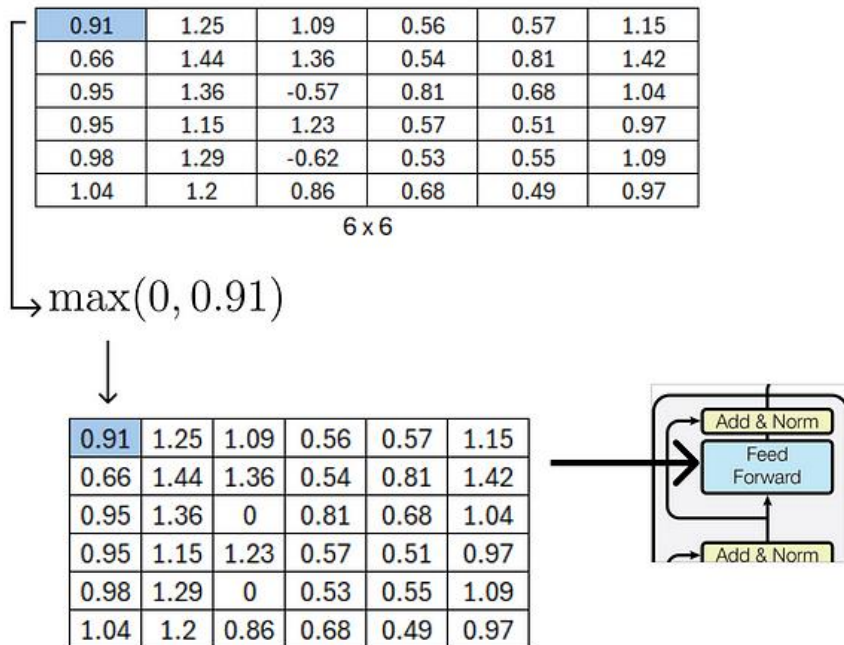
=

|      |      |       |      |      |      |
|------|------|-------|------|------|------|
| 0.91 | 1.25 | 1.09  | 0.56 | 0.57 | 1.15 |
| 0.66 | 1.44 | 1.36  | 0.54 | 0.81 | 1.42 |
| 0.95 | 1.36 | -0.57 | 0.81 | 0.68 | 1.04 |
| 0.95 | 1.15 | 1.23  | 0.57 | 0.51 | 0.97 |
| 0.98 | 1.29 | -0.62 | 0.53 | 0.55 | 1.09 |
| 1.04 | 1.2  | 0.86  | 0.68 | 0.49 | 0.97 |

6 x 6

After calculating the linear layer, we need to pass it through the ReLU layer and use its formula.

$$\text{ReLU}(x) = \max(0, x)$$



### **Step 10 — Adding and Normalizing Again**

Once we obtain the resultant matrix from feed forward network, we have to add it to the matrix that is obtained from previous add and norm step, and then normalizing it using the row wise mean and standard deviation.

Matrix from Feed Forward Network

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| 0.91 | 1.25 | 1.09 | 0.56 | 0.57 | 1.15 |
| 0.66 | 1.44 | 1.36 | 0.54 | 0.81 | 1.42 |
| 0.95 | 1.36 | 0    | 0.81 | 0.68 | 1.04 |
| 0.95 | 1.15 | 1.23 | 0.57 | 0.51 | 0.97 |
| 0.98 | 1.29 | 0    | 0.53 | 0.55 | 1.09 |
| 1.04 | 1.2  | 0.86 | 0.68 | 0.49 | 0.97 |

+

Matrix from Previous Add and Norm Step

|      |      |       |       |       |       |
|------|------|-------|-------|-------|-------|
| 1.51 | 1.14 | -0.62 | 0.11  | -1.4  | -0.25 |
| 2.02 | 0.54 | -0.78 | -0.06 | -1.02 | -0.19 |
| 1.54 | 1.08 | -0.82 | 0.03  | -1.36 | 0.11  |
| 1.64 | 1.09 | -0.23 | -0.26 | -1.4  | -0.26 |
| 1.65 | 0.99 | -0.34 | -0.16 | -1.47 | -0.21 |
| 1.16 | 1.37 | -0.69 | 0.23  | -1.52 | -0.09 |

=

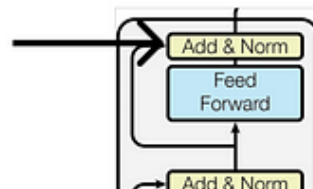
|      |      |       |      |       |      |
|------|------|-------|------|-------|------|
| 2.42 | 2.39 | 0.47  | 0.67 | -0.83 | 0.9  |
| 2.68 | 1.98 | 0.58  | 0.48 | -0.21 | 1.23 |
| 2.49 | 2.44 | -0.82 | 0.84 | -0.68 | 1.15 |
| 2.59 | 2.24 | 1     | 0.31 | -0.89 | 0.71 |
| 2.63 | 2.28 | -0.34 | 0.37 | -0.92 | 0.88 |
| 2.2  | 2.57 | 0.17  | 0.91 | -1.03 | 0.88 |

→

| Mean   | Std      |
|--------|----------|
| 1.0033 | 1.103534 |
| 1.1233 | 1.214349 |
| 0.9033 | 1.301837 |
| 0.9933 | 1.289055 |
| 0.8167 | 1.306016 |
| 0.95   | 1.320773 |

=

|      |      |       |       |       |       |
|------|------|-------|-------|-------|-------|
| 1.28 | 1.26 | -0.48 | -0.3  | -1.66 | -0.09 |
| 1.28 | 0.71 | -0.45 | -0.53 | -1.1  | 0.09  |
| 1.22 | 1.18 | -1.32 | -0.05 | -1.22 | 0.19  |
| 1.24 | 0.97 | 0.01  | -0.53 | -1.46 | -0.22 |
| 1.39 | 1.12 | -0.89 | -0.34 | -1.33 | 0.05  |
| 0.95 | 1.23 | -0.59 | -0.03 | -1.5  | -0.05 |



The output matrix of this add and norm step will serve as the query and key matrix in one of the multi-head attention mechanisms present in the decoder part, which you can easily understand by tracing outward from the add and norm to the decoder section.

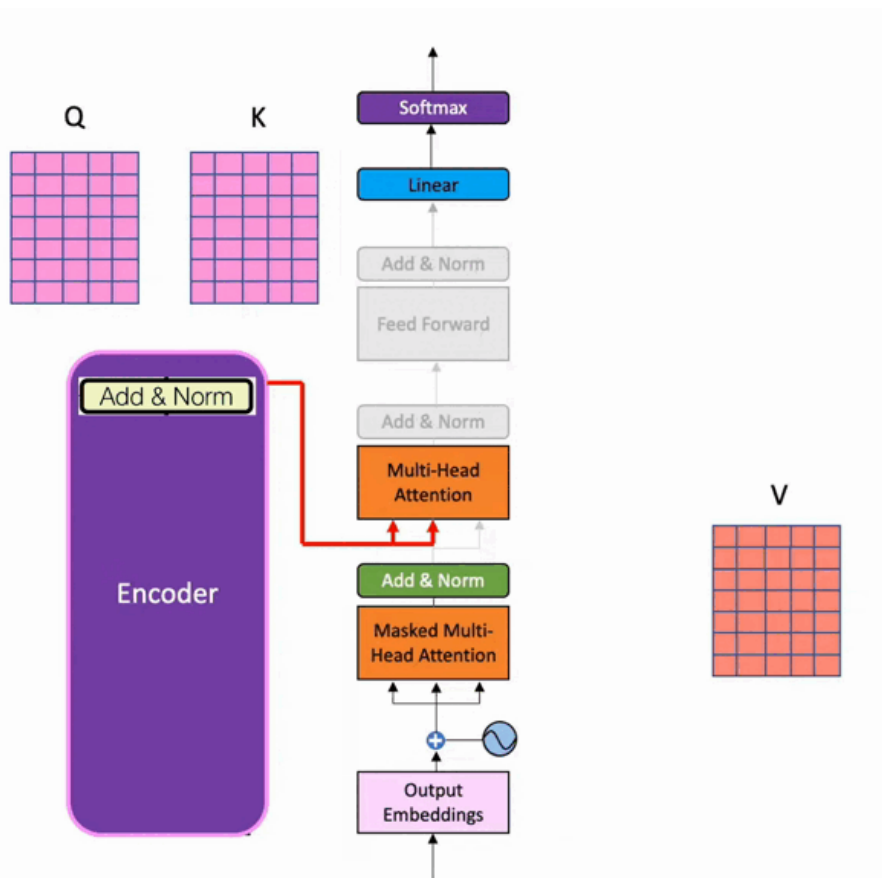
### Step 11 — Decoder Part

The good news is that up until now, we have calculated **Encoder part**, all the steps that we have performed, from encoding our dataset to passing our matrix through the feedforward network, are unique. It means we haven't calculated them before. But from now on, all the upcoming steps that is the remaining architecture of the transformer (**Decoder part**) are going to involve similar kinds of matrix multiplications.

We won't be calculating the entire decoder because most of its portion contains similar calculations to what we have already done in the encoder. Calculating the decoder in detail would only make the blog lengthy due to repetitive steps. Instead, we only need to focus on the calculations of the input and output of the decoder.

When training, there are two inputs to the decoder. One is from the encoder, where the output matrix of the last add and norm layer serves as the **query** and **key** for the second multi-head attention layer in the decoder part. Below is the visualization of it



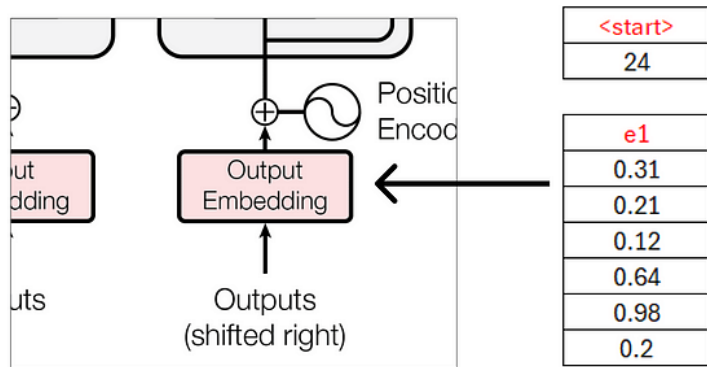


While the value matrix comes from the decoder after the first **add and norm** step. The second input to the decoder is the predicted text. If you remember, our input to the encoder is `when you play game of thrones` so the input to the decoder is the predicted text, which in our case is `you win or you die`. But the predicted input text needs to follow a standard wrapping of tokens that make the transformer aware of where to start and where to end.

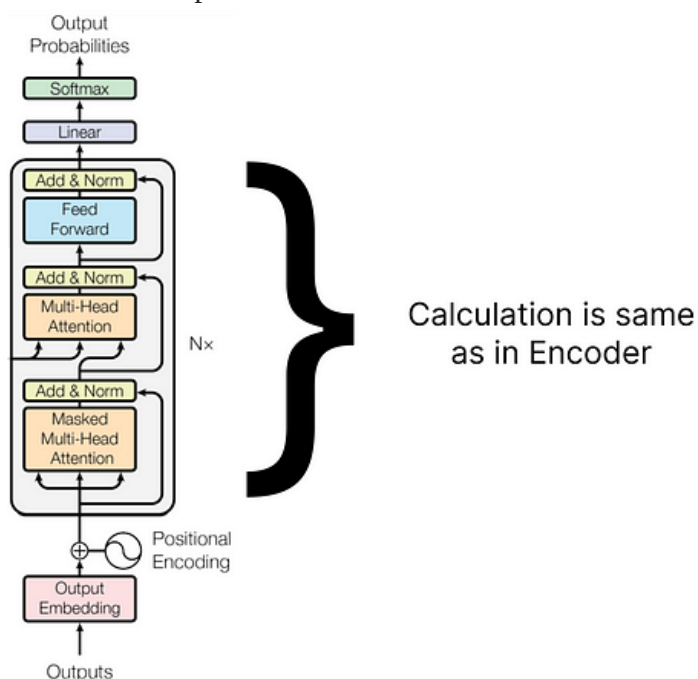
Encoder Input → `When you play game of thrones`

Decoder Input → `<start> you win or you die <end>`

Where `<start>` and `<end>` are two new tokens being introduced. Moreover, the decoder takes one token as an input at a time. It means that `<start>` will be served as an input, and `you` must be the predicted text for it.



As we already know, these embeddings are filled with random values, which will later be updated during the training process. Compute rest of the blocks in the same way that we computed earlier in the encoder part.



### Step 12 — Understanding Mask Multi Head Attention

In a Transformer, the masked multi-head attention is like a spotlight that a model uses to focus on different parts of a sentence. It's special because it doesn't let the model cheat by looking at words that come later in the sentence. This helps the model understand and generate sentences step by step, which is important in tasks like talking or translating words into another language.

Suppose we have the following input matrix, where each row represents a position in the sequence, and each column represents a feature:

$$\text{Input Matrix} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Now, let's understand the masked multi-head attention components having two heads:

1. **Linear Projections (Query, Key, Value):** Assume the linear projections for each head: **Head 1:  $Wq1, Wk1, Wv1$**  and **Head 2:  $Wq2, Wk2, Wv2$**
2. **Calculate Attention Scores:** For each head, calculate attention scores using the dot product of Query and Key, and apply the mask to prevent attending to future positions.
3. **Apply Softmax:** Apply the softmax function to obtain attention weights.
4. **Weighted Summation (Value):** Multiply the attention weights by the Value to get the weighted sum for each head.
5. **Concatenate and Linear Transformation:** Concatenate the outputs from both heads and apply a linear transformation.

**Let's do a simplified calculation:**

Assuming two conditions

- $Wq1 = Wk1 = Wv1 = Wq2 = Wk2 = Wv2 = I$ , the identity matrix.
- $Q=K=V=\text{Input Matrix}$

**Head 1:**

$$Q_1 = K_1 = V_1 = \text{Input Matrix}$$

$$A_1 = Q_1 \cdot K_1^T$$

$$A_1 = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

$$A_1 = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 5 & 0 \\ 7 & 8 & 9 \end{bmatrix} \quad (\text{Masked})$$

$$W_1 = \text{softmax}(A_1)$$

$$O_1 = W_1 \cdot V_1$$

**Head 2:**

$$Q_2 = K_2 = V_2 = \text{Input Matrix}$$

$$A_2 = Q_2 \cdot K_2^T$$

$$A_2 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 5 & 6 \\ 0 & 0 & 9 \end{bmatrix} \quad (\text{Masked})$$

$$W_2 = \text{softmax}(A_2)$$

$$O_2 = W_2 \cdot V_2$$

**Concatenate and Linear Transformation:**

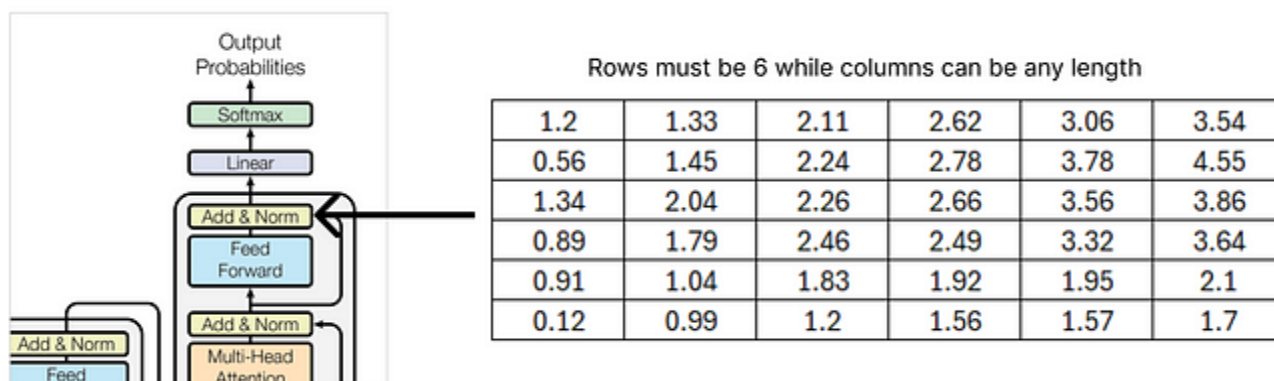
$$\text{Concatenate}([O_1, O_2])$$

(Apply Learnable Linear Transformation)

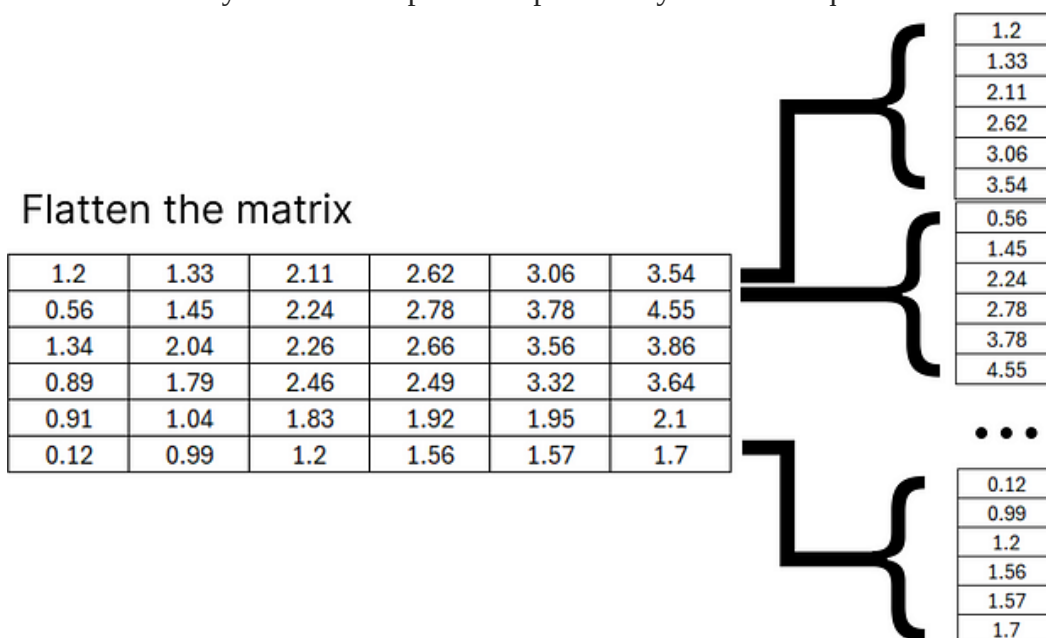
The concatenation step combines the outputs from the two attention heads into a single set of information. Imagine you have two friends who each give you advice on a problem. Concatenating their advice means putting both pieces of advice together so that you have a more complete view of what they suggest. In the context of the transformer model, this step helps capture different aspects of the input data from multiple perspectives, contributing to a richer representation that the model can use for further processing.

### Step 13 — Calculating the Predicted Word

The output matrix of the last add and norm block of the decoder must contain the same number of rows as the input matrix, while the number of columns can be any. Here, we work with 6.



The last **add and norm block** resultant matrix of the decoder must be flattened in order to match it with a linear layer to find the predicted probability of each unique word in our dataset (corpus).



This flattened layer will be passed through a linear layer to compute the **logits** (scores) of each unique word in our dataset.

$$\text{Linear Layer} = X \cdot W$$

No Bias

Flatten Layer (Row Matrix)

|     |      |      |     |     |      |      |     |
|-----|------|------|-----|-----|------|------|-----|
| 1.2 | 1.33 | 2.11 | ... | 1.2 | 1.56 | 1.57 | 1.7 |
|-----|------|------|-----|-----|------|------|-----|

1 x n (our case n is 36)

X

Linear set of weights

|      |       |        |     |      |
|------|-------|--------|-----|------|
| I    | drink | things | ... | He   |
| 1    | 2     | 3      | ... | 23   |
| 0.28 | 0.72  | 0.14   |     | 0.11 |
| 0.1  | 0.5   | 0.93   |     | 0.37 |
| 0.77 | 0.23  | 0.17   | ... | 0.51 |
| 0.23 | 0.62  | 0.99   |     | 0.76 |
| 0.75 | 0.76  | 0.42   | ... | 0.26 |
| 0.02 | 0.52  | 0.47   |     | 0.24 |
| 0.58 | 0.9   | 0.88   | ... | 0.54 |
| ...  | ...   | ...    |     | ...  |
| 0.59 | 0.16  | 0.05   |     | 0.87 |
| 0.35 | 0.02  | 0.84   |     | 0.67 |

n x m  
(m is 23 Vocab Size)

logits =

|      |       |        |     |     |     |     |
|------|-------|--------|-----|-----|-----|-----|
| I    | drink | things | ... | you | ... | He  |
| 1    | 2     | 3      | ... | 17  | ... | 23  |
| 1.14 | 2.3   | 1.15   | ... | 5.4 | ... | 2.3 |

Once we obtain the logits, we can use the **softmax** function to normalize them and find the word that contains the highest probability.

logits =

|      |       |        |     |     |     |     |
|------|-------|--------|-----|-----|-----|-----|
| I    | drink | things | ... | you | ... | He  |
| 1    | 2     | 3      | ... | 17  | ... | 23  |
| 1.14 | 2.3   | 1.15   | ... | 5.4 | ... | 2.3 |

↓

Applying softmax

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

↓

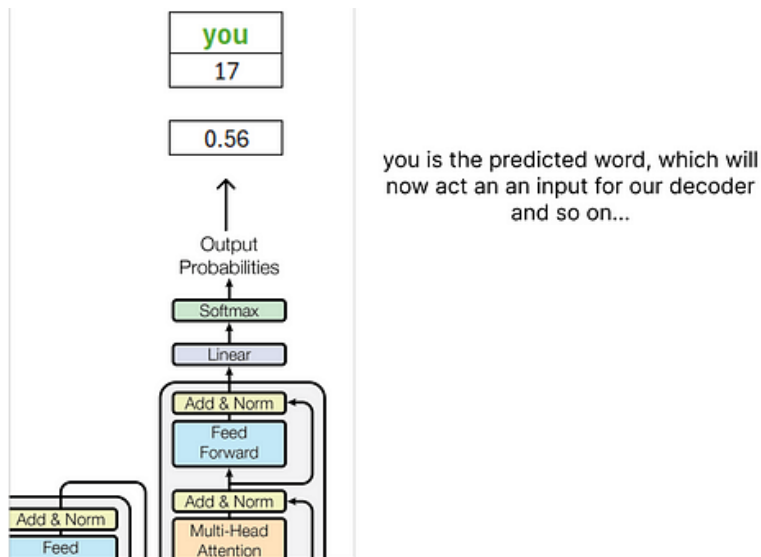
Probabilities =

|      |       |        |     |      |     |      |
|------|-------|--------|-----|------|-----|------|
| I    | drink | things | ... | you  | ... | He   |
| 1    | 2     | 3      | ... | 17   | ... | 23   |
| 0.21 | 0.05  | 0.001  | ... | 0.56 | ... | 0.12 |

↑

highest Probability

So based on our calculations, the predicted word from the decoder is you.



This predicted word 'you', will be treated as the input word for the decoder, and this process continues until the <end> token is predicted.

